ASSIGNMENT – 4

OBJECT ORIENTED PROGRAMMING USING C++

DESIGNING REUSABLE CLASS HIERARCHIES

Name	SHAHANA K
Register Number	192524032
Course Code	DSA0101
Subject	Object Oriented Programming Using C++
Assignment	Assignment 4 – Designing Reusable Class Hierarchies
Total Marks	50

Compilation command

g++ -std=c++17 -Wall -Wextra -pedantic task.cpp -o task

./task

For the memory diagnostic in Task 4, use an AddressSanitizer-enabled build:

g++ -std=c++17 -Wall -Wextra -g -fsanitize=address -fno-omit-frame-pointer task4.cpp -o task4

./task4

The screenshots supplied show Tasks 1–6. Some small text in the first three screenshots is low-resolution. The answers below use the visible requirements and standard C++ rules; check the exact original class field names against your portal before final upload.

Core concepts needed throughout the assignment

Concept	Easy meaning	Why it matters here
Class / object	A class is a blueprint; an object is one real instance.	All tasks model staff, people, accounts, carts and vehicles.
Encapsulation	Keep data inside a class and expose safe public functions.	Avoids duplicated and directly exposed data.
Inheritance	A derived class reuses and extends a base class.	Professor, Student, SeniorAccount and Car.
Polymorphism	One base interface can call different derived implementations.	Virtual pay(), rate() and charge().
Virtual function	A function selected at run time using the object's real type.	Ensures derived behaviour through base pointers.
Pure virtual function	A virtual function declared with = 0.	Makes a class abstract, e.g. Vehicle::charge().

TASK 1 – CLASS DESIGN REVIEW

Bloom's Level: K5 – Evaluate | Total: 6 Marks

Task 1.1 – Diagnose the design (2 Marks)

The original design uses separate Professor, LabAssistant and GuestFaculty classes. Each class repeats fields such as name and an identifier, and repeats the pay() operation. The design is difficult to maintain because a change to common staff information must be made in several classes.

• Repeated fields: name and staff identification data appear in multiple classes.

• Repeated rules: the basic salary and allowance calculation is repeated.

• Public data members: any part of the program can change an employee's details without validation.

• Inconsistent names: one class may use id while another uses code for the same concept.

• Unused fields: some staff categories contain fields that do not apply to them.

• Magic number 0.42: the allowance percentage is embedded directly in the calculation.

• Poor extensibility: adding a new staff category requires another independent class and repeated code.

• No common collection: the program cannot easily process all staff through one common interface.

Task 1.2 – Repair the design (3 Marks)

Create an abstract base class Staff. Put common information in Staff and make pay() a pure virtual function. Each derived class overrides pay() with its own salary rule. Keep data protected or private; private data with public accessors is generally safer.

Proposed class hierarchy

Staff (abstract)

name, staffId, basicPay

virtual pay() = 0

/ | \

/ | \

Professor LabAssistant GuestFaculty

research allowance hours, rate

pay() pay() pay()

```
#include <iostream>
#include <iomanip>
#include <memory>
#include <string>
#include <vector>
using namespace std;
class Staff {
protected:
string name;
int staffId;
double basicPay;
public:
Staff(string n, int id, double basic)
: name(n), staffId(id), basicPay(basic) {}
virtual double pay() const = 0; // pure virtual function
virtual void display() const {
cout << name << " (" << staffId << ") : "
<< fixed << setprecision(2) << pay() << '\n';
}
virtual ~Staff() = default; // safe deletion through Staff*
};
class Professor : public Staff {
double researchAllowance;
public:
Professor(string n, int id, double basic, double research)
: Staff(n, id, basic), researchAllowance(research) {}
```

```
    double pay() const override {
    return basicPay + basicPay * 0.42 + researchAllowance;
    }
    };
    class LabAssistant : public Staff {
    double allowance;
    public:
LabAssistant(string n, int id, double basic, double extra)
: Staff(n, id, basic), allowance(extra) {}
    double pay() const override {
    return basicPay + basicPay * 0.42 + allowance;
    }
    };
    class GuestFaculty : public Staff {
    double hours;
    double hourlyRate;
    public:
GuestFaculty(string n, int id, double h, double r)
: Staff(n, id, 0.0), hours(h), hourlyRate(r) {}
    double pay() const override {
    return hours * hourlyRate;
    }
    };
    int main() {
    vector<unique_ptr<Staff>> staff;
staff.push_back(make_unique<Professor>("Anita", 101, 50000, 8000));
staff.push_back(make_unique<LabAssistant>("Bala", 102, 30000, 2000));
staff.push_back(make_unique<GuestFaculty>("Charu", 103, 40, 750));
    for (const auto& person : staff)
person->display();
    }
```

Illustrative output: Professor Anita: 79,000.00; LabAssistant Bala: 44,600.00; GuestFaculty Charu: 30,000.00. The program prints each name, ID and calculated pay. (Exact formatting follows the code.)

Task 1.3 – Design defence (1 Mark; within 100 words)

The hierarchy makes adding a new staff category easier because common data and the pay() interface already exist in Staff. A new class only inherits Staff and implements its own pay() function. Existing code that stores Staff pointers and calls pay() does not need to change. This follows the Open–Closed Principle: the design is open for extension but closed for unnecessary modification.

Alternative concept and complexity analysis

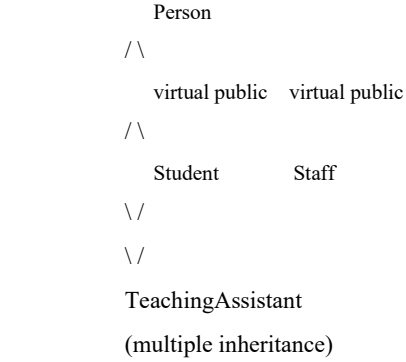
An alternative is one Staff class with a category field and a switch statement inside pay(). This is initially simple, but every new category requires editing the switch and often adding more fields to every object. The hierarchy uses one virtual call, which is O(1), just like selecting a switch branch for a fixed number of categories. The main benefit is maintainability and extension, not a better asymptotic running time.

TASK 2 – FROM REQUIREMENT TO HIERARCHY

University People Management | 8 Marks

Task 2.1 – Identify relationships (2 Marks)

The university has Person as the common base. Student and Staff are both kinds of Person, so each uses public inheritance from Person. A TeachingAssistant is both a Student and a Staff member, so it uses multiple inheritance from Student and Staff. Student and Staff use virtual public inheritance from Person to ensure TeachingAssistant contains only one shared Person subobject.



Task 2.2 – Resolve the diamond (2 Marks)

Without virtual inheritance, TeachingAssistant would contain two Person subobjects: one through Student and another through Staff. The compiler would not know which name or age to use when the program refers to them through TeachingAssistant. Virtual inheritance creates one shared Person base subobject.

Task 2.3 – Class declarations and access (3 Marks)

```
#include <iostream>
#include <string>
using namespace std;
class Person {
public:
string name;
Person() : name("Unknown") {}
explicit Person(string n) : name(n) {}
};
class Student : virtual public Person {
public:
double cgpa = 0.0;
};
class Staff : virtual public Person {
public:
string department = "General";
};
class TeachingAssistant : public Student, public Staff {
public:
TeachingAssistant(string n, double g, string d) {
name = n; // one shared Person::name
cgpa = g;
department = d;
}
};
int main() {
```

```
TeachingAssistant ta("Kavya", 8.7, "CSE");
    cout << "Name: " << ta.name << '\n';
    cout << "CGPA: " << ta.cgpa << '\n';
    cout << "Department: " << ta.department << '\n';
    }
```

Output

Name: Kavya

CGPA: 8.7

Department: CSE

In a production design, Person should have a constructor and TeachingAssistant should explicitly initialise the virtual base Person. The following is the more correct constructor form:

```
    class Person {
    public:
string name;
explicit Person(string n = "Unknown") : name(n) {}
    };
    class Student : virtual public Person { public: double cgpa = 0; };
    class Staff : virtual public Person { public: string department = "General"; };
    class TeachingAssistant : public Student, public Staff {
    public:
TeachingAssistant(string n, double g, string d)
: Person(n), Student(), Staff() {
cgpa = g;
department = d;
    }
    };
```

Task 2.4 – Access table (1 Mark)

Member declared in Person

Student public inheritance

Staff public inheritance

public

public in Student

public in Staff

protected

protected in Student

protected in Staff

private

not directly accessible in derived class

not directly accessible in derived class

Private Person members still exist inside the Person subobject, but Student and Staff cannot access them directly.

They must use public or protected Person member functions.

TASK 3 – CONSTRUCTORS IN DERIVED CLASSES AND

MEMBER CLASSES

Order of Construction and Destruction | 9 Marks

Task 3.1 – Pass values to base constructors (3 Marks)

A derived constructor passes base-class values using the member-initializer list. The base class is constructed before the derived-class members and before the derived constructor body.

Task 3.2 – Member classes / composition (3 Marks)

Student contains an Address object. This is composition: the Address member is part of the Student object and its lifetime is controlled by Student. It is initialised in the initializer list.

Task 3.3 – Trace the order (2 Marks)

Construction order: virtual base classes first; then direct base classes in the order they appear in the class declaration; then data members in the order they are declared in the class; finally the constructor body. Destruction occurs in exactly the reverse order.

Complete Code – Student and Address

```
#include <iostream>
#include <string>
using namespace std;
class Person {
protected:
string name;
int age;
public:
Person(const string& n, int a) : name(n), age(a) {}
};
class Address {
string city;
int pin;
public:
Address(const string& c, int p) : city(c), pin(p) {}
string str() const {
return city + "-" + to_string(pin);
}
};
class Student : public Person {
long long regNo;
Address home;
public:
Student(const string& n, int a, long long r,
const string& city, int pin)
: Person(n, a), regNo(r), home(city, pin) {}
void print() const {
cout << regNo << " " << name << " (" << age
<< ") " << home.str() << '\n';
}
};
int main() {
Student s("Kavya", 19, 192311001, "Salem", 636001);
s.print();
}
```

Expected output

192311001 Kavya (19) Salem-636001

Task 3.4 – Experiment: Engine, Vehicle and Car (1 Mark)

```
#include <iostream>
using namespace std;
struct Engine {
Engine()  { cout << "Engine "; }
~Engine() { cout << "~Engine "; }
};
struct Vehicle {
Vehicle()  { cout << "Vehicle "; }
~Vehicle() { cout << "~Vehicle "; }
};
struct Car : Vehicle {
Engine e;
Car()  { cout << "Car "; }
~Car() { cout << "~Car "; }
};
int main() {
{
Car c;
cout << "| ";
}
cout << '\n';
}
```

Expected output:

```
Vehicle Engine Car | ~Car ~Engine ~Vehicle
```

Vehicle is the base class, so it is constructed first. Engine is a member of Car and is constructed next. The Car constructor body runs last. On leaving the block, the Car destructor body runs first, then member Engine is destroyed, and finally base Vehicle is destroyed.

What changes if virtual is removed from a diamond?

If Student and Staff no longer virtually inherit Person, TeachingAssistant contains two Person subobjects. This increases object size and makes unqualified access to Person members ambiguous. With virtual inheritance, there is one shared Person subobject. Use sizeof(TeachingAssistant) to measure the implementation-specific size; do not assume a fixed byte count because padding and compiler ABI affect it.

Dynamic Address pointer alternative

If Student stores Address* home instead of Address home, the constructor must allocate it with new and the destructor must delete it. A deep-copy constructor and copy-assignment operator are needed to avoid two Student objects owning the same pointer. A safer modern alternative is std::unique_ptr, which automatically releases memory and prevents accidental copying unless cloning is explicitly implemented.

```
#include <memory>
class StudentWithPointer : public Person {
long long regNo;
std::unique_ptr<Address> home;
public:
StudentWithPointer(const string& n, int a, long long r,
const string& city, int pin)
```

: Person(n, a), regNo(r),

home(make_unique<Address>(city, pin)) {}

```
// unique_ptr automatically deletes Address.
// Copying is disabled unless a deliberate deep-copy is written.
};
```

Complexity: direct member construction is O(1) and needs no separate allocation. A dynamic member adds an allocation/deallocation cost and stores a pointer (typically one machine word) in the Student object. It is useful when optional or polymorphic ownership is needed, but unnecessary for a simple fixed Address value.

TASK 4 – POINTERS TO DERIVED CLASSES: FORENSICS

7 Marks

Task 4.1 – Locate the defects (2 Marks)

- rate() in Account is not virtual. Calling it through Account* selects Account::rate() at compile time, so the SeniorAccount rate of 4.0 is hidden.
- Account has no virtual destructor. Deleting a SeniorAccount through Account* is undefined behaviour; the derived destructor may not run, so its dynamically allocated history array may leak.
- Account copy = SeniorAccount(); copies only the Account base part. This is object slicing: the SeniorAccount-specific portion is discarded.
- SeniorAccount owns a raw dynamic array. If copied shallowly, two objects could point to the same array and cause double deletion. Ownership needs clear copy/move rules or a standard container.

Task 4.2 and 4.3 – Corrected program

```
#include <iostream>
#include <memory>
#include <vector>
using namespace std;
class Account {
public:
virtual double rate() const { return 3.5; }
virtual ~Account() { cout << "Account destroyed\n"; }
};
class SeniorAccount : public Account {
double* history;
public:
SeniorAccount() : history(new double[100]{}) {}
double rate() const override { return 4.0; }
~SeniorAccount() override {
delete[] history;
cout << "SeniorAccount destroyed; history released\n";
}
};
int main() {
vector<unique_ptr<Account>> accounts;
accounts.push_back(make_unique<Account>());
accounts.push_back(make_unique<SeniorAccount>());
for (const auto& a : accounts)
cout << a->rate() << '\n';
}
```

Expected rates: 3.5 then 4.0. When the vector is destroyed, the SeniorAccount destructor runs before the Account destructor for the senior object. The base destructor is virtual, so deletion through the base pointer is safe. unique_ptr automatically owns and releases each object.

Sanitizer evidence

Build the original defective program with AddressSanitizer and run it. A sanitizer may report a leak or invalid deletion depending on compiler and runtime. Undefined behaviour is not guaranteed to produce the same visible message on every platform. Add a print statement to each destructor to observe which destructor is called. In the repaired version, the derived and base destructors both run.

Task 4.4 – Two prevention rules (1 Mark)

• If a base class is intended to be deleted through a base pointer, give it a virtual destructor.

• Use virtual on polymorphic operations and use override in derived classes so signature mistakes are caught by the compiler.

• Prefer unique_ptr/shared_ptr or standard containers over manually managed raw owning pointers.

• Use references or pointers for polymorphic objects; do not copy a derived object into a base object when its derived behaviour must be retained.

Predict the output – static versus dynamic binding

```
#include <iostream>
using namespace std;
struct Account {
double rate() const { return 3.5; }
virtual double vrate() const { return 3.5; }
virtual ~Account() = default;
};
struct SeniorAccount : Account {
double rate() const { return 4.0; } // hides, does not override
double vrate() const override { return 4.0; }
};
int main() {
SeniorAccount s;
Account* p = &s;
Account copy = s;
cout << p->rate() << ' ' << p->vrate()
<< ' ' << copy.vrate() << '\n';
}
```

Exact output

3.5 4 3.5

First value: p->rate() is non-virtual, so the compile-time pointer type Account* selects Account::rate(), giving 3.5. Second value: p->vrate() is virtual, so run-time dispatch uses the real object type SeniorAccount, giving 4.0. Third value: copy is a separate Account object created by slicing; it has no SeniorAccount part, so copy.vrate() gives 3.5.

unique_ptr alternative and complexity

Store accounts in vector> and call virtual rate() through each pointer. Processing n accounts is O(n); each rate call is O(1). The vector stores one smart pointer per account plus each separately allocated object. It prevents manual delete mistakes and supports virtual dispatch, but it cannot by itself fix a non-virtual function or prevent slicing if the programmer deliberately copies objects by value.

TASK 5 – POINTERS TO OBJECTS AND THE this POINTER

7 Marks

Task 5.1 – Four ways to reach an object (3 Marks)

Access method

Example

Meaning

Direct object

c.total()

Use object name and dot operator.

Pointer

p->total()

Pointer stores address; arrow accesses member.

Reference

```
Cart& r = c; r.total()
```

Reference is an alias for an existing object.

Dynamic pointer

```
Cart* q = new Cart; q->total(); delete q;
```

Object is dynamically allocated and must be released.

Complete TODO 1 and TODO 2

```
#include <iomanip>
#include <iostream>
#include <string>
using namespace std;
class Cart {
string items[10];
double prices[10];
int n = 0;
double discount = 0; // percentage
public:
Cart& add(const string& item, double price) {
if (n < 10) {
items[n] = item;
prices[n] = price;
++n;
}
return *this; // return current object by reference
}
Cart& setDiscount(double discount) {
this->discount = discount; // member = parameter
return *this;
}
double total() const {
double t = 0;
for (int i = 0; i < n; ++i)
t += prices[i];
return t * (100 - discount) / 100.0;
```

```
    }
    };
    int main() {
    Cart c;                         // direct object
c.add("Rice", 60).add("Dal", 120).setDiscount(10);
    Cart* p = &c;                   // pointer to existing object
p->add("Oil", 180);
    Cart& ref = c;                  // reference to same object
    cout << "Through reference: " << ref.total() << '\n';
    Cart* dynamicCart = new Cart;       // dynamically allocated object
dynamicCart->add("Milk", 50);
    cout << "Dynamic cart: " << dynamicCart->total() << '\n';
delete dynamicCart;
    cout << fixed << setprecision(2)
<< "Main cart total: " << p->total() << '\n';
    }
```

Expected output

Through reference: 324

Dynamic cart: 50

Main cart total: 324.00

The main cart contains Rice (60), Dal (120) and Oil (180). The subtotal is 360. A 10% discount gives 360 × 90 / 100 = 324.

What exactly is this?

Inside a non-static member function, this is a pointer to the object on which the function was called. *this means the actual current object. Returning *this as Cart& returns that same object by reference, allowing chained calls such as c.add(...).add(...).setDiscount(...).

What if add() returned Cart instead of Cart&?

Returning Cart by value creates and returns a copy. Chained calls may operate on temporary copies rather than the original object, and extra copying can occur. Returning Cart& avoids the copy and allows the chain to keep modifying the same cart.

Task 5.2 – Find the highest total using an array of pointers (2 Marks)

```
    #include <iostream>
    using namespace std;
    // Assume Cart has a public total() const function.
    int main() {
    Cart a, b, c;
a.add("Rice", 60);
b.add("Rice", 60).add("Oil", 180);
c.add("Dal", 120).add("Milk", 50);
    Cart* carts[] = { &a, &b, &c };
int count = sizeof(carts) / sizeof(carts[0]);
int best = 0;
    for (int i = 1; i < count; ++i) {
    if (carts[i]->total() > carts[best]->total())
best = i;
```

```
    }
    cout << "Highest cart total: " << carts[best]->total() << '\n';
    }
```

The loop examines each cart once, so the time complexity is O(n). The pointer array uses O(n) extra space for n pointers. Each Cart object exists separately; the array only stores its address.

Task 5.3 – Safety, lifetime and use (2 Marks)

Method

Lifetime / safety

Best use

Direct object

Automatic lifetime; destroyed at scope end.

Normal local objects.

Pointer to existing object

Does not own the object; can become dangling if object dies.

Optional access, polymorphism, APIs.

Reference

Must refer to an existing object; cannot be reseated.

Required non-null alias or function parameter.

new/delete pointer

Manual lifetime; forgetting delete leaks memory.

Low-level ownership only; prefer smart pointers.

Alternative: vector instead of pointer array

A vector stores the Cart objects themselves. Finding the maximum still takes O(n) time. It stores full Cart objects rather than one pointer per cart, so its storage is larger when Cart is large. When the vector grows and reallocates, pointers, references and iterators to its elements may become invalid. A vector> keeps the Cart objects at stable separately allocated addresses while the vector moves only the smart pointers.

TASK 6 – DESIGN CHALLENGE: VEHICLE RENTAL FLEET

13 Marks

Stage 1 – Interpret the problem (2 Marks)

Entities: Vehicle, Bicycle, MotorVehicle, TwoWheeler, Car, ElectricCar, and optionally ElectricBus. Common data: registration number and daily rate. Motor vehicles additionally have fuel/charge level and service due date. Cars add seating capacity. Electric cars add battery capacity and an electricity tariff.

Testable requirements:

- Every vehicle has a unique registration number and a non-negative daily rate.
- The system can list all registered vehicles.
- The system can rent an available vehicle and mark it unavailable.
- The system can return a rented vehicle and calculate any late fee.
- Each vehicle type calculates its rental charge according to its own rule.
- A new vehicle type can be added by deriving from Vehicle without editing existing derived classes.
- The fleet stores mixed vehicle types through base-class pointers and releases all owned objects safely.

Stage 2 – Propose the hierarchy (2 Marks)

Vehicle (abstract)

registrationNo, dailyRate,

rented, rent(), return()

charge(days) = 0

|

+------------+-------------+

| |

Bicycle MotorVehicle

fuelOrChargeLevel

serviceDueDate

/ \

/ \

TwoWheeler Car

seats

|

ElectricCar

batteryCapacity

pricePerKWh

ElectricBus can be added later as another Vehicle-derived class.

Vehicle to Bicycle and Vehicle to MotorVehicle are hierarchical inheritance branches. TwoWheeler and Car inherit from MotorVehicle. ElectricCar inherits from Car in this proposed model, because it is a specialised car. Each derived class uses public inheritance. A separate Battery or ServiceRecord class could be used as a member class if the design needs more detailed composition.

Stage 3 – Critical design decisions (2 Marks)

• Keep common registration, daily rate and rental status in Vehicle.

• Make Vehicle abstract using pure virtual charge(int days).

• Make charge() virtual so the real vehicle type determines the rental charge.

• Give Vehicle a virtual destructor because objects are deleted through Vehicle pointers.

• Keep fuel, service date, seats and battery capacity only in the classes that need them.

• Store the fleet in vector> to own mixed vehicle types safely.

Stage 4 – Class declarations and pseudocode (4 Marks)

Rental pseudocode: Find the vehicle by registration number. If it exists and is not rented, mark it rented and confirm the rental. Otherwise report unavailable/not found. On return, find the rented vehicle, calculate days used, call its virtual charge(days), add a late fee if returned after the due date, mark it available and print the total.

IF vehicle.registrationNo == requestedRegNo

IF vehicle.isRented == false

vehicle.rent()

PRINT "Rental successful"

ELSE

PRINT "Vehicle is already rented"

END IF

ELSE

PRINT "Vehicle not found"

END IF

ON RETURN:

rentalDays = actualReturnDate - startDate

baseCharge = vehicle.charge(rentalDays)

lateFee = max(0, lateDays) * lateFeePerDay

total = baseCharge + lateFee

vehicle.returnVehicle()

PRINT total

Stage 5 – Complete C++ implementation (3 Marks)

```
#include <iostream>
#include <memory>
#include <string>
#include <vector>
#include <algorithm>
using namespace std;
class Vehicle {
protected:
string regNo;
double dailyRate;
bool rented = false;
public:
Vehicle(string reg, double rate)
: regNo(reg), dailyRate(rate) {}
virtual double charge(int days) const = 0;
bool rent() {
if (rented) return false;
rented = true;
return true;
}
void returnVehicle() { rented = false; }
bool isRented() const { return rented; }
string registration() const { return regNo; }
virtual void display() const {
cout << regNo << " | daily rate: " << dailyRate
<< " | " << (rented ? "Rented" : "Available") << '\n';
}
virtual ~Vehicle() = default;
};
class Bicycle : public Vehicle {
public:
Bicycle(string reg, double rate) : Vehicle(reg, rate) {}
double charge(int days) const override {
return days * dailyRate;
}
};
class MotorVehicle : public Vehicle {
protected:
```

```
    double fuelLevel;
string serviceDueDate;
    public:
MotorVehicle(string reg, double rate, double fuel, string due)
: Vehicle(reg, rate), fuelLevel(fuel), serviceDueDate(due) {}
    };
    class TwoWheeler : public MotorVehicle {
    public:
TwoWheeler(string reg, double rate, double fuel, string due)
: MotorVehicle(reg, rate, fuel, due) {}
    double charge(int days) const override {
    return days * dailyRate + 50.0; // fixed handling fee
    }
    };
    class Car : public MotorVehicle {
    protected:
int seats;
    public:
    Car(string reg, double rate, double fuel, string due, int s)
: MotorVehicle(reg, rate, fuel, due), seats(s) {}
    double charge(int days) const override {
    return days * dailyRate + 200.0; // cleaning/service fee
    }
    };
    class ElectricCar : public Car {
    double batteryCapacity;
    double pricePerKWh;
    public:
ElectricCar(string reg, double rate, double fuel, string due,
int s, double battery, double tariff)
: Car(reg, rate, fuel, due, s),
batteryCapacity(battery), pricePerKWh(tariff) {}
    double charge(int days) const override {
    double energyCost = batteryCapacity * pricePerKWh;
    return days * dailyRate + energyCost;
    }
    };
    // New type: existing classes above do not need modification.
    class ElectricBus : public Vehicle {
int passengerCapacity;
    double batteryKWh;
    double pricePerKWh;
    public:
ElectricBus(string reg, double rate, int passengers,
    double battery, double tariff)
: Vehicle(reg, rate), passengerCapacity(passengers),
batteryKWh(battery), pricePerKWh(tariff) {}
    double charge(int days) const override {
```

```
    return days * dailyRate + batteryKWh * pricePerKWh;
    }
    };
    int main() {
    vector<unique_ptr<Vehicle>> fleet;
fleet.push_back(make_unique<Bicycle>("BI-101", 100));
fleet.push_back(make_unique<TwoWheeler>("TW-202", 250, 80, "2027-01-15"));
fleet.push_back(make_unique<Car>("CA-303", 1500, 60, "2027-02-01", 5));
fleet.push_back(make_unique<ElectricCar>(
"EV-404", 1800, 0, "2027-02-10", 5, 60, 8));
fleet.push_back(make_unique<ElectricBus>(
"EB-505", 3000, 40, 120, 9));
    cout << "FLEET LIST\n";
    for (const auto& v : fleet) v->display();
    cout << "\nRENTAL AND RETURN DEMONSTRATION\n";
    for (const auto& v : fleet) {
    if (v->rent()) {
    cout << v->registration() << " rented. ";
int days = 2;
    double base = v->charge(days);
int lateDays = 1;
    double lateFee = lateDays * 150.0;
    cout << "Base charge = " << base
<< ", late fee = " << lateFee
<< ", total = " << base + lateFee << '\n';
v->returnVehicle();
    }
    }
    cout << "\nAFTER RETURN\n";
    for (const auto& v : fleet) v->display();
    }
```

The daily rates and fee amounts in this demonstration are sample business rules chosen to make the program executable. Replace them if your instructor specifies different tariffs. The key design requirement is that each derived class supplies its own charge() implementation.

Illustrative output and interpretation

The program lists five vehicle objects, including the newly added ElectricBus. It then rents each vehicle, calculates two days of base charge, adds a one-day late fee of 150, and returns the vehicle. Finally, it lists the fleet again; every vehicle should show Available. Exact numeric totals are calculated by the formulas in the code.

Alternative concept and complexity analysis

A single Vehicle class with a type code and switch statement can calculate charges. Both a switch with a fixed number of types and a virtual function call are O(1) per charge call. The switch design can waste space because one object may contain fields used only by another type. It also requires editing the central switch and possibly several tests whenever a new vehicle type is added. The abstract hierarchy stores only relevant derived data and lets a new type add its own charge() implementation without changing existing classes. The hierarchy is more extensible and easier to maintain.

Final comparison and important rules

Rule	Reason
Use public inheritance for an “is-a” relationship.	A Student is a Person; a Car is a MotorVehicle.
Use composition for a “has-a” relationship.	A Student has an Address.
Use virtual inheritance for a shared diamond base.	Avoids duplicate Person subobjects.
Use a virtual destructor in polymorphic base classes.	Makes deletion through base pointers safe.
Use override in derived classes.	The compiler checks that a virtual function is correctly overridden.
Use pure virtual functions for abstract interfaces.	Prevents creating incomplete base objects.
Initialise bases and members in initializer lists.	Required for base/member construction and efficient initialisation.
Return *this as a reference for chaining.	Avoids making a copy of the current object.
Prefer smart pointers for ownership.	Automatic cleanup reduces leaks and double deletion.

Viva questions with short answers

1. What is inheritance? A mechanism by which a derived class acquires and extends the properties and functions of a base class.
2. What is polymorphism? One interface can represent different behaviours depending on the actual object type.
3. Why is a virtual destructor important? It ensures the derived destructor is called when deleting a derived object through a base pointer.
4. What is a pure virtual function? A virtual function declared with = 0; it makes the class abstract.
5. What is object slicing? Copying a derived object into a base object by value, which discards the derived-only part.
6. What is a virtual base class? A base inherited with virtual inheritance so a most-derived object shares one base subobject.
7. What is composition? A class owns another object as a data member.
8. What is the order of construction? Virtual bases, direct bases, members in declaration order, then constructor body.
9. What is the order of destruction? The reverse of construction: destructor body, members in reverse order, bases in reverse order.
10. What does this point to? The object whose non-static member function is currently executing.
11. What does *this mean? The current object itself, as an lvalue.
12. Why return Cart& from add()? It returns the same cart without copying and supports method chaining.
13. What is the difference between . and ->? Dot accesses a member through an object; arrow accesses a member through a pointer.
14. What is the difference between pointer and reference? A pointer stores an address and can be null/reassigned; a reference is an alias that must be initialised.

15. What does O(n) mean in the cart search? The number of operations grows linearly with the number of carts.

Submission checklist

• Put each independent program in its own .cpp file (for example task1.cpp through task6.cpp).

• Compile with -std=c++17 -Wall -Wextra -pedantic and fix warnings where possible.

• Run each program and capture genuine output screenshots.

• Run the original and repaired Task 4 versions with AddressSanitizer if available; include the actual report.

• Draw or reproduce the class diagrams and access table in the report if your lecturer expects handwritten or formal UML.

• Create a GitHub repository and upload source files, screenshots, README.md and this report.

• Add your real GitHub URL to the cover page or README before submitting.

• Practise the viva questions and be ready to explain every virtual function, pointer, constructor and destructor.

Academic accuracy note: Some outputs in this report are derived directly from the C++ code and language rules. The assignment asks students to predict, compile and compare. For full evidence marks, write your prediction before running the program, then attach the output from your own compiler and explain any difference. Do not label illustrative output as a personally executed result.